Modul Praktik MuJoCo LiDAR: Memahami dan Menggunakan LiDAR di Arena Virtual

Oct 12, 2026 · @Admin Tirta Hakim P

Tujuan, prasyarat, dan alur modul

Setelah menyelesaikan modul ini, Anda bisa menjelaskan cara kerja LiDAR dan memakainya di arena virtual MuJoCo, tanpa sensor LiDAR fisik sama sekali. Seluruh praktik berjalan di laptop biasa (CPU saja, tanpa GPU).

Tujuan pembelajaran

1. Menjelaskan prinsip time-of-flight, titik hasil pindai (point cloud), dan parameter utama LiDAR (FoV, resolusi, jangkauan).
2. Menjelaskan bagaimana LiDAR "palsu" dihasilkan dengan ray casting di MuJoCo, dan apa bedanya dengan sensor nyata.
3. Memasang `mujoco-lidar` dan memindai scene sederhana dengan `MjLidarWrapper`.
4. Membangun arena virtual (dinding, rintangan, robot) dalam MJCF dan memindainya.
5. Memilih dan mengubah pola scan (satu garis, grid, Velodyne, Ouster, dsb.).
6. Mengubah jarak mentah menjadi titik 3D, lalu memvisualisasikan dan menganalisisnya.
7. Memakai LiDAR untuk robot bergerak yang menghindari rintangan.

Prasyarat

- Python 3.10 sampai 3.13 (README repo menyebut rentang ini; skrip modul diuji dengan Python 3.12).
- Dasar Python dan NumPy: array, slicing, boolean mask.
- Dasar trigonometri (sin, cos) dan konsep koordinat 3D.
- Tidak perlu GPU, tidak perlu ROS, tidak perlu pengalaman MuJoCo sebelumnya.

Alur modul

Bagian	Isi	Estimasi waktu
1	Konsep dasar LiDAR	20 menit
2	LiDAR di simulasi (ray casting)	15 menit
3	Instalasi dan cek lingkungan	15 menit
4	Mengenal repo dan API	20 menit

Bagian	Isi	Estimasi waktu
Praktik 1	Scan pertama	20 menit
Praktik 2	Membangun arena	30 menit
Praktik 3	Pola scan dan parameter	30 menit
Praktik 4	Visualisasi dan analisis	40 menit
Praktik 5	Robot bergerak di arena	45 menit
Latihan	Soal mandiri dan kunci	40 menit

Estimasi waktu adalah perkiraan saya untuk pemula; sesuaikan dengan kecepatan Anda.

Cara memakai modul. Buat satu folder kerja bernama `lidar_lab`. Semua file praktik (`arena.xml`, `p1_scan_pertama.py`, dst.) disimpan di folder itu dan dijalankan dari dalam folder itu.

Tips membaca: hampir setiap penjelasan diikuti paragraf **Versi mudah**. Jika rumus atau istilahnya terasa berat, baca versi mudahnya dulu, lalu kembali ke penjelasan lengkap.

Bagian 1: Cara kerja LiDAR

LiDAR (Light Detection and Ranging) mengukur jarak dengan menembakkan pulsa laser, lalu menghitung berapa lama pantulannya kembali. Satu pulsa menghasilkan satu angka jarak; ribuan pulsa ke arah berbeda menghasilkan "peta" titik 3D yang disebut point cloud.

1.1 Prinsip time-of-flight

Cahaya menempuh jarak ke objek lalu kembali, jadi waktu tempuh dibagi dua:

```
d = \frac{c \cdot \Delta t}{2}
```

Dengan c = 299.792.458 m/s dan Δt waktu antara pulsa keluar dan pantulan diterima. Contoh: pantulan kembali setelah 20 ns berarti d = (3×10^8 × 20×10^-9) / 2 = 3 m. Jadi sensor nyata harus mengukur waktu di orde nanodetik. Di simulasi, kita langsung mendapat d tanpa mengukur waktu.

Versi mudah: bayangkan Anda berteriak di lembah lalu menghitung berapa lama gema kembali. Makin lama gemanya, makin jauh tebingnya. LiDAR melakukan hal yang sama, tetapi dengan cahaya laser. Karena cahaya sangat cepat, alatnya harus punya "stopwatch" super teliti. Di simulasi kita tidak perlu stopwatch: komputer langsung tahu jaraknya.

1.2 Dari satu sinar ke satu titik 3D

Setiap sinar punya dua sudut dalam bingkai koordinat sensor: azimut θ (menyamping, keliling sumbu vertikal) dan elevasi φ (naik-turun). Dengan jarak r, posisi titik pantul adalah:

```
x = r \cos\varphi \cos\theta, \quad y = r \cos\varphi \sin\theta, \quad z = r \sin\varphi
```

Rumus ini terbukti cocok dengan keluaran `get_hit_points()` di MuJoCo-LiDAR (diuji di Praktik 1), jadi Anda bisa memakainya untuk memahami apa yang dilakukan library.

Versi mudah: pegang senter. θ adalah seberapa jauh Anda memutar senter ke kiri atau kanan, φ adalah seberapa tinggi Anda mengangkatnya, dan r adalah jarak ke benda yang tersorot. Rumus di atas hanya mengubah "arah + jarak" menjadi posisi (x, y, z), seperti mengubah petunjuk "jalan 5 langkah ke arah timur laut" menjadi titik di peta.

1.3 Dari satu titik ke satu pindaian (scan)

LiDAR mekanis berputar sehingga θ menyapu 360°, sementara beberapa laser tersusun vertikal dengan φ berbeda-beda. Satu putaran penuh disebut satu frame atau satu scan. Hasilnya sekumpulan (θ, φ, r) yang bisa diubah menjadi point cloud.

Versi mudah: seperti menyapukan senter ke seluruh ruangan sambil mencatat titik-titik yang terang. Satu putaran sapuan lengkap adalah satu scan, dan kumpulan titik catatannya disebut point cloud.

1.4 Parameter yang menentukan kualitas LiDAR

Parameter	Arti	Dampak praktis
FoV horizontal	Lebar sudut θ yang tercakup (mis. 360°)	Seberapa luas sekeliling terlihat
FoV vertikal	Rentang sudut φ (mis. -25° sampai +2°)	Bisa melihat lantai, atau hanya setinggi sensor
Jumlah garis (channel)	Banyaknya laser vertikal (16, 32, 64, 128)	Kerapatan titik vertikal
Resolusi sudut	Selisih sudut antar sinar	Detail objek kecil dan jauh
Jangkauan maksimum	Jarak terjauh yang terukur	Sinar tanpa pantulan dalam jangkauan = tidak ada data
Laju pindai	Berapa frame per detik (mis. 10 Hz)	Seberapa cepat peta diperbarui

Versi mudah: FoV adalah seberapa lebar pandangan mata. Jumlah garis dan resolusi adalah seberapa rapat "jaring" sinar yang disebar (makin rapat, makin detail). Jangkauan adalah sejauh apa mata masih bisa melihat. Laju pindai adalah berapa kali per detik mata "berkedip" memperbarui gambar.

1.5 Keterbatasan sensor nyata (untuk dibandingkan dengan simulasi)

- Noise pengukuran: jarak selalu punya galat kecil.
- Permukaan sulit: kaca dan benda sangat mengkilap bisa tembus atau memantul ke arah lain; benda hitam menyerap cahaya.
- Cuaca: hujan, kabut, dan debu menghasilkan titik palsu.
- Gerak: saat sensor berputar sambil robot bergerak, satu frame terdistorsi.

Cek pemahaman. (a) Pantulan kembali 40 ns; berapa jaraknya? (b) Mengapa LiDAR 64 channel memberi gambaran vertikal lebih rapat daripada 16 channel? Jawaban ada di bagian Latihan.

Bagian 2: LiDAR di simulasi tanpa sensor fisik

Di simulasi, "laser" adalah garis lurus matematis yang ditembakkan dari titik sensor ke arah tertentu. Teknik ini disebut ray casting: komputer mencari permukaan pertama yang dipotong garis itu, lalu menghitung jaraknya. Tidak ada cahaya, waktu tempuh, atau perangkat keras yang terlibat, jadi sensor fisik tidak diperlukan.

Versi mudah: di simulasi, "laser" hanyalah garis lurus imajiner. Komputer bertanya: "garis ini pertama kali menyentuh apa, dan berapa jauh?" Mirip permainan tembak-tembakan: sistem cukup menghitung peluru kena apa dan di jarak berapa, tanpa alat sungguhan.

2.1 Alur satu kali scan di MuJoCo-LiDAR

1. Anda menaruh sebuah `site` (titik penanda dengan posisi dan orientasi) di model MJCF. Site ini menjadi "lensa" LiDAR dan ikut bergerak bersama body induknya.
2. Anda menyiapkan daftar sudut sinar: array `theta` (azimut) dan `phi` (elevasi), satu pasang per sinar. Inilah yang disebut pola scan.
3. `MjLidarWrapper.trace_rays(data, theta, phi)` membaca pose site saat itu, memutar semua arah sinar ke bingkai dunia, lalu menembakkannya ke semua geom di scene.
4. Untuk tiap sinar, library mengembalikan jarak ke geom pertama yang kena. Pada backend CPU, ini memakai fungsi bawaan MuJoCo `mj_multiRay`.
5. Dari jarak dan sudut, Anda (atau `get_hit_points()`) menghitung titik 3D untuk membentuk point cloud.

Versi mudah: (1) pasang "kamera" di robot (site), (2) tentukan ke arah mana saja sinar ditembakkan (θ dan φ), (3) tembak (`trace_rays`), (4) terima daftar jarak, (5) ubah jarak menjadi titik-titik untuk digambar.

2.2 Apa yang disimulasikan dan apa yang tidak

Aspek	Sensor nyata	MuJoCo-LiDAR (versi 0.3.5 yang saya uji)
Jarak per sinar	Diukur lewat time-of-flight	Dihitung geometris, tepat
Titik 3D	Dihitung dari jarak dan sudut	`get_hit_points()` atau hitung sendiri
Noise	Selalu ada	Tidak ada di API yang saya periksa; tambahkan sendiri jika perlu
Intensitas pantulan	Ada	Tidak ada di API yang saya periksa
Kaca, cuaca, benda gelap	Memengaruhi hasil	Tidak dimodelkan: semua geom dianggap memantul
Pola scan	Tetap per model sensor	Bebas; tersedia preset Velodyne, Ouster, Livox, dll.

Implikasinya: simulasi ini bagus untuk memahami geometri, menguji algoritma, dan latihan integrasi, tetapi hasilnya lebih "bersih" daripada sensor nyata. Jika nanti algoritma Anda dipindah ke robot fisik, siapkan penanganan noise.

Versi mudah: LiDAR di simulasi itu "sempurna dan bersih", sedangkan LiDAR nyata "agak berisik". Cocok untuk belajar dan menguji ide, tetapi jangan kaget kalau sensor asli tidak sebersih ini.

2.3 Pilihan backend

Satu antarmuka (`MjLidarWrapper`), empat mesin hitung. Angka kecepatan di bawah diambil dari README repo (bergantung perangkat keras).

Backend	Kecepatan (sinar/detik)	Perlu GPU	Catatan
`cpu`	sekitar 9 juta	Tidak	Memakai `mj_multiRay` MuJoCo; dipakai di seluruh modul ini
`taichi`	sekitar 62 juta	Ya	Mendukung mesh dan heightfield
`jax`	sekitar 231 juta	Ya	Cocok untuk simulasi batch (MJX)
`warp`	sekitar 100 juta	Ya	Mendukung mesh dinamis dan scene batch

Penting: nilai bawaan `backend` pada konstruktor adalah `"taichi"` , jadi di komputer tanpa Taichi/GPU Anda harus menulis `backend="cpu"` secara eksplisit.

Bagian 3: Instalasi dan cek lingkungan

Backend CPU hanya butuh `mujoco` , `mujoco-lidar` , `numpy` , dan `matplotlib` . Langkah di bawah diuji di Python 3.12 dengan `mujoco-lidar` 0.3.5 dan MuJoCo 3.15.0.

3.1 Buat folder dan lingkungan virtual

Linux atau macOS:

```
mkdir lidar_lab && cd lidar_lab
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install mujoco mujoco-lidar numpy matplotlib
```

Windows (PowerShell):

```
mkdir lidar_lab; cd lidar_lab
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install mujoco mujoco-lidar numpy matplotlib
```

README repo juga menyarankan `uv` (`uv add mujoco-lidar`). Itu setara; modul ini memakai `pip` karena paling umum.

3.2 Skrip cek instalasi

Simpan sebagai `p0_cek.py`, lalu jalankan `python p0_cek.py`:

```
import mujoco
import mujoco_lidar
from mujoco_lidar import MjLidarWrapper, scan_gen

print('MuJoCo      :', mujoco.__version__)
print('mujoco_lidar:', mujoco_lidar.__version__)
print('Pola tersedia:', [n for n in dir(scan_gen) if
n.startswith(('generate', 'create'))])
```

Keluaran yang diharapkan: dua baris versi dan daftar fungsi seperti `generate_HDL64`, `generate_grid_scan_pattern`, `create_lidar_single_line`. Jika muncul `ModuleNotFoundError`, lingkungan virtual belum aktif atau `pip install` gagal (lihat bagian Troubleshooting).

3.3 Opsional: unduh repo untuk contoh bawaan

Repo berisi contoh siap pakai (viewer, robot Unitree Go2/G1, ROS). Anda tidak wajib memakainya untuk modul ini, tetapi berguna sebagai bahan belajar lanjutan:

```
git clone https://github.com/discoverse-dev/MuJoCo-LiDAR.git
cd MuJoCo-LiDAR
pip install -e ".[examples]"
python examples/example_native.py --backend cpu
```

Perintah `example_native.py --backend cpu` tertulis di README repo dan membuka MuJoCo viewer, jadi perlu layar (tidak jalan di server tanpa display). Perintah `pip install -e ".[examples]"` adalah padanan pip dari `uv sync --extra examples` di README; saya tidak menguji langkah clone ini di sandbox, jadi jika gagal, ikuti panduan resmi INSTALLATION.md.

Bagian 4: Mengenal repo dan API MuJoCo-LiDAR

4.1 Peta folder repo

Folder atau file	Isi	Kapan dibuka
`src/mujoco_lidar/`	Kode library: `MjLidarWrapper`, generator pola scan, backend	Saat ingin tahu cara kerja di dalam
`examples/`	Contoh: `example_native.py`, `example_mjcf.py`, `example_string.py`, robot Go2, G1, T800, ROS	Setelah Praktik 1 sampai 5
`models/`	Scene MJCF contoh (mis. `demo.xml`)	Untuk bahan membuat arena sendiri
`docs/en/`	Panduan: `INSTALLATION.md`, `USAGE.md`, `API.md`, dst.	Referensi resmi
`benchmarks/`, `tests/`	Uji kecepatan dan uji otomatis	Opsional

4.2 Kelas inti: `MjLidarWrapper`

```
MjLidarWrapper(
    mj_model,                # mujoco.MjModel
    site_name,               # nama site LiDAR di model
    backend='taichi',        # 'cpu' | 'taichi' | 'jax' | 'warp'
    cutoff_dist=100.0,       # jarak maksimum (meter)
    args={},                 # opsi khusus backend
)
```

Anggota	Fungsi
`trace_rays(data, theta, phi)`	Menembakkan semua sinar; mengembalikan array jarak (meter) sepanjang jumlah sinar
`get_distances()`	Mengambil kembali jarak dari scan terakhir
`get_hit_points()`	Titik 3D hasil scan terakhir dalam bingkai koordinat sensor
`sensor_position`, `sensor_rotation`	Posisi (3,) dan matriks rotasi (3×3) site pada scan terakhir

Anggota	Fungsi
`trace_rays_batch(...)`	Versi batch (untuk backend JAX/Warp)

Opsi backend CPU yang paling berguna (`args`):

- `bodyexclude` : ID body yang diabaikan sinar, supaya LiDAR tidak "melihat" badan robotnya sendiri. Contoh: `{'bodyexclude': model.body('robot').id}` .
- `geomgroup` : array 6 elemen `uint8` untuk memilih grup geom mana yang terlihat oleh sinar.

4.3 Generator pola scan

Semua mengembalikan pasangan array `(theta, phi)` dalam radian.

Fungsi	Pola
`create_lidar_single_line(horizontal_resolution=360, horizontal_fov=2π)`	Satu garis horizontal (seperti LiDAR 2D)
`generate_grid_scan_pattern(num_ray_cols, num_ray_rows, theta_range, phi_range)`	Grid sudut buatan sendiri
`generate_HDL64()` , `generate_vlp32()` , `generate_os128()` , `generate_airy96()`	Preset sensor nyata (Velodyne, Ouster, RoboSense)
`LivoxGenerator(pattern='mid360').sample_ray_angles()`	Pola Livox non-repetitif (modul `mujoco_lidar.scan_gen_livox_ti`)

4.4 Tiga temuan dari pengujian yang perlu Anda ingat

Ini hasil percobaan langsung saya pada versi 0.3.5 (backend CPU), dan akan dipakai di seluruh praktik:

1. Sinar yang tidak mengenai apa pun mengembalikan jarak `-1.0` , bukan `cutoff_dist` atau 0. Selalu saring dengan `r > 0` .
2. `get_hit_points()` berada di bingkai koordinat sensor, bukan dunia. Untuk koordinat dunia: `titik_dunia = titik_lokal @ sensor_rotation.T + sensor_position` .
3. Untuk sinar yang meleset, baris pada `get_hit_points()` bernilai (0, 0, 0). Jangan memplot baris itu tanpa menyaringnya.

Versi mudah: (1) nilai -1 berarti "sinar ini tidak kena apa-apa", jadi abaikan. (2) Titik dari `get_hit_points()` adalah posisi "dilihat dari sudut pandang robot"; supaya bisa dibandingkan dengan peta arena, ubah ke sudut pandang dunia dengan rumus yang tadi.

(3) Titik (0, 0, 0) hanyalah tanda kosong untuk sinar yang meleset, bukan benda sungguhan.

Untuk membaca kode secara langsung: `print(inspect.getsource(MjLidarWrapper.trace_rays))` (modul `inspect`) menampilkan implementasinya di komputer Anda. Referensi resmi: API.md dan USAGE.md.

Praktik 1: Scan LiDAR pertama

Tujuan: memindai satu kotak dengan LiDAR satu garis, lalu membuktikan sendiri bahwa jarak dan titik 3D cocok dengan teori di Bagian 1.

Langkah:

1. Pastikan lingkungan virtual aktif dan `p0_cek.py` berjalan.
2. Simpan kode berikut sebagai `p1_scan_pertama.py` di folder `lidar_lab`.
3. Jalankan `python p1_scan_pertama.py`.

```
import numpy as np
import mujoco
from mujoco_lidar import MjLidarWrapper, scan_gen

# 1) Scene: lantai, satu kotak 1 m x 1 m x 1 m di x = 2 m, dan site LiDAR di
titik (0, 0, 0.5)
XML = '''
<mujoco>
  <worldbody>
    <geom name='lantai' type='plane' size='10 10 0.1'/>
    <body name='kotak' pos='2 0 0.5'>
      <geom name='kotak_geom' type='box' size='0.5 0.5 0.5'/>
    </body>
    <site name='lidar_site' pos='0 0 0.5'/>
  </worldbody>
</mujoco>
'''
model = mujoco.MjModel.from_xml_string(XML)
data = mujoco.MjData(model)
mujoco.mj_forward(model, data)          # hitung posisi semua objek sebelum
scan

# 2) Buat LiDAR (backend cpu WAJIB ditulis; bawaannya taichi)
lidar = MjLidarWrapper(model, site_name='lidar_site', backend='cpu',
cutoff_dist=20.0)
```

```
# 3) Pola scan: 1 garis horizontal, 360 sinar, mengelilingi sensor
theta, phi = scan_gen.create_lidar_single_line(horizontal_resolution=360)

# 4) Tembakkan sinar
r = lidar.trace_rays(data, theta, phi)
print('jumlah sinar      :', len(r))
print('sinar yang kena   :', int((r > 0).sum()))
print('sinar yang meleset:', int((r < 0).sum()), '(nilainya -1)')

# 5) Jarak ke kotak tepat di depan (theta paling dekat 0)
i = int(np.argmin(np.abs(theta)))
print(f'jarak sinar ke depan: {r[i]:.3f} m  (harapan: 1.5 m, karena sisi
depan kotak di x = 1.5)')

# 6) Hitung titik 3D sendiri dari r, theta, phi, lalu bandingkan dengan
get_hit_points()
hit = r > 0
manual = np.stack([r*np.cos(phi)*np.cos(theta),
                   r*np.cos(phi)*np.sin(theta),
                   r*np.sin(phi)], axis=1)
pts = lidar.get_hit_points()
print('rumus sendiri == get_hit_points():', np.allclose(manual[hit],
pts[hit], atol=1e-4))
```

Keluaran yang diharapkan (hasil uji saya):

```
jumlah sinar      : 360
sinar yang kena   : 36
sinar yang meleset: 324 (nilainya -1)
jarak sinar ke depan: 1.500 m  (harapan: 1.5 m, karena sisi depan kotak di x
= 1.5)
rumus sendiri == get_hit_points(): True
```

Penjelasan hasil:

- Hanya 36 dari 360 sinar yang kena. Kotak selebar 1 m pada jarak 1,5 m terlihat selebar sekitar 37° (2 × arctan(0,5/1,5)), dan resolusi sinar sekitar 1°, jadi wajar sekitar 36 sinar.
- Sinar horizontal tidak pernah menyentuh lantai karena sejajar dengan lantai. Inilah sebabnya LiDAR satu garis hanya "melihat" objek setinggi sensor.
- Baris terakhir `True` membuktikan rumus konversi di Bagian 1.2 sama dengan yang dihitung library.

Versi mudah: robot menyorotkan senter ke 360 arah. Hanya 36 sorotan yang mengenai kotak, karena kotaknya sempit dan sisanya mengarah ke ruang kosong. Sorotan yang sejajar lantai tidak pernah menyentuh lantai, jadi LiDAR satu garis hanya melihat benda

yang setinggi dirinya. Angka `True` di akhir artinya: rumus buatan Anda sama persis dengan hasil library.

Eksperimen cepat (5 menit):

- Ubah `pos='2 0 0.5'` pada kotak menjadi `pos='4 0 0.5'`. Prediksikan dulu jarak ke depan dan jumlah sinar yang kena, lalu jalankan dan cocokkan.
- Ubah `cutoff_dist=20.0` menjadi `1.0`. Apa yang terjadi pada jumlah sinar yang kena? (Petunjuk: kotak berada di 1,5 m.)

Praktik 2: Membangun arena virtual sendiri

Tujuan: membuat arena 8 m × 8 m berdinding, berisi tiga rintangan dan satu robot ber-LiDAR, lalu memindainya dari dua posisi berbeda.

2.1 Konsep MJCF yang dipakai

Elemen	Fungsi di arena
`geom`	Bentuk fisik yang bisa kena sinar: `plane`, `box`, `cylinder`, `sphere`, dll.
`body`	Pembungkus geom yang punya posisi; bisa digerakkan
`site`	Titik tanpa bentuk fisik. Di sini dipakai sebagai posisi LiDAR
`mocap='true'`	Body yang posisinya diatur lewat kode (`data.mocap_pos`), bukan lewat fisika. Cocok untuk memindahkan robot sederhana

Catatan geometri: dinding berupa box tipis. Dinding utara berpusat di y = 4,05 dengan setengah tebal 0,05, sehingga sisi dalamnya tepat di y = 4,0. Karena itu jarak ke dinding dari titik (0, 0) harus 4,0 m, dan ini dipakai sebagai uji kebenaran.

2.2 Langkah

1. Simpan MJCF berikut sebagai `arena.xml`.
2. Simpan skrip di bawahnya sebagai `p2_arena.py`.
3. Jalankan `python p2_arena.py`.
4. Buka `p2_tampak_atas.png` dan cocokkan bentuk titik dengan denah arena.

```
<mujoco model='arena_lidar'>
  <worldbody>
    <light pos='0 0 6' dir='0 0 -1'/>
    <geom name='lantai' type='plane' size='6 6 0.1' rgba='0.8 0.8 0.8 1'/>

    <!-- dinding arena 8 m x 8 m (sisi dalam di +-4 m) -->
    <geom name='dinding_utara'   type='box' pos='0 4.05 0.5'  size='4.1 0.05
0.5' rgba='0.3 0.3 0.8 1'/>
    <geom name='dinding_selatan' type='box' pos='0 -4.05 0.5' size='4.1 0.05
0.5' rgba='0.3 0.3 0.8 1'/>
    <geom name='dinding_timur'   type='box' pos='4.05 0 0.5'  size='0.05 4.1
0.5' rgba='0.3 0.3 0.8 1'/>
    <geom name='dinding_barat'   type='box' pos='-4.05 0 0.5' size='0.05 4.1
0.5' rgba='0.3 0.3 0.8 1'/>

    <!-- tiga rintangan -->
    <body name='rintangan_kotak' pos='2 1 0.4'>
      <geom name='kotak' type='box' size='0.4 0.4 0.4' rgba='0.9 0.3 0.2 1'/>
    </body>
    <body name='rintangan_silinder' pos='-1.5 2 0.5'>
      <geom name='silinder' type='cylinder' size='0.3 0.5' rgba='0.2 0.8 0.3
1'/>
    </body>
    <body name='rintangan_bola' pos='1 -2.5 0.35'>
      <geom name='bola' type='sphere' size='0.35' rgba='0.9 0.8 0.2 1'/>
    </body>

    <!-- robot: body mocap (digerakkan lewat kode, bukan lewat fisika) -->
    <body name='robot' mocap='true' pos='0 0 0.15'>
      <geom name='badan_robot' type='box' size='0.2 0.15 0.08' rgba='0.1 0.1
0.1 1'/>
      <site name='lidar_site' pos='0 0 0.2' size='0.03' rgba='1 0 0 1'/>
    </body>
  </worldbody>
</mujoco>
```

```
import numpy as np
import mujoco
import matplotlib
matplotlib.use('Agg')                    # simpan ke file; hapus baris ini
jika ingin jendela interaktif
import matplotlib.pyplot as plt
from mujoco_lidar import MjLidarWrapper, scan_gen

model = mujoco.MjModel.from_xml_path('arena.xml')
```

```
data = mujoco.MjData(model)

robot_id = model.body('robot').id
mocap_id = model.body_mocapid[robot_id]

def taruh_robot(x, y, yaw):
    data.mocap_pos[mocap_id] = [x, y, 0.15]
    data.mocap_quat[mocap_id] = [np.cos(yaw/2), 0, 0, np.sin(yaw/2)]   #
quaternion (w, x, y, z)
    mujoco.mj_forward(model, data)

lidar = MjLidarWrapper(model, site_name='lidar_site', backend='cpu',
                       cutoff_dist=30.0, args={'bodyexclude': robot_id})
theta, phi = scan_gen.create_lidar_single_line(360)

def scan_dunia():
    r = lidar.trace_rays(data, theta, phi)
    hit = r > 0
    lokal = lidar.get_hit_points()[hit]
    dunia = lokal @ np.asarray(lidar.sensor_rotation).T +
lidar.sensor_position
    return r, hit, dunia

# Pose 1: tengah arena
taruh_robot(0, 0, 0)
r, hit, dunia = scan_dunia()
for nama, ang in [('depan (+x)', 0), ('kiri (+y)', 90), ('belakang', 180),
('kanan (-y)', -90)]:
    i = int(np.argmin(np.abs(theta - np.radians(ang))))
    print(f'pose 1, {nama:11s}: {r[i]:.2f} m')

# Pose 2: pojok barat daya, menghadap timur
taruh_robot(-3, -3, 0)
r2, hit2, dunia2 = scan_dunia()
i0 = int(np.argmin(np.abs(theta)))
print(f'pose 2, depan (+x)   : {r2[i0]:.2f} m  (harapan 7.00 = 4 - (-3))')

# Gambar tampak atas kedua scan
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(dunia[:, 0],  dunia[:, 1],  s=6, label='pose 1 (0, 0)')
ax.scatter(dunia2[:, 0], dunia2[:, 1], s=6, label='pose 2 (-3, -3)')
ax.set_aspect('equal'); ax.grid(True); ax.legend()
ax.set_title('Point cloud tampak atas (koordinat dunia)')
fig.savefig('p2_tampak_atas.png', dpi=100)
print('gambar disimpan: p2_tampak_atas.png')
```

Keluaran yang diharapkan (hasil uji saya):

```
pose 1, depan (+x) : 4.00 m
pose 1, kiri (+y)  : 4.00 m
pose 1, belakang   : 4.00 m
pose 1, kanan (-y) : 4.00 m
pose 2, depan (+x)   : 7.00 m  (harapan 7.00 = 4 - (-3))
gambar disimpan: p2_tampak_atas.png
```

2.3 Hal yang perlu dipahami dari kode

- `mujoco.mj_forward(...)` wajib dipanggil setelah mengubah `mocap_pos`, supaya posisi site diperbarui sebelum scan. Tanpa ini, LiDAR memindai dari posisi lama.
- `bodyexclude=robot_id` membuat sinar mengabaikan badan robot sendiri. Tanpa ini, sinar yang berawal di dalam atau dekat geom robot bisa mengenai badannya.
- Kuaternion MuJoCo berformat (w, x, y, z). Rotasi pada sumbu z sebesar yaw adalah (cos(yaw/2), 0, 0, sin(yaw/2)).
- Titik dunia dihitung dengan `lokal @ R.T + posisi`, sesuai temuan di Bagian 4.4.

Versi mudah: `mj_forward` itu seperti "refresh layar" setelah Anda menggeser robot, supaya komputer tahu posisi barunya. `bodyexclude` berarti "jangan hitung badan sendiri sebagai rintangan". Kuaternion hanyalah cara komputer menyimpan arah hadap robot. Robot mocap itu seperti bidak catur yang kita geser dengan tangan lewat kode.

2.4 Opsional: melihat arena di viewer

Jika komputer Anda punya layar, jalankan `python -m mujoco.viewer --mjcf=arena.xml` untuk melihat arena tiga dimensi. Perintah ini tidak saya uji di sandbox (butuh display), tetapi merupakan viewer standar bawaan paket `mujoco`.

Eksperimen: tambahkan satu `body` rintangan baru berisi `geom` box `size='0.3 0.3 0.3'` pada `pos='-2 -1 0.3'` di dalam `worldbody`, pindahkan robot ke (-3, -1) dengan yaw 0, dan perkirakan jarak ke depan sebelum menjalankan scan. (Jawaban di Latihan.)

Praktik 3: Pola scan dan parameter LiDAR

Tujuan: membandingkan beberapa pola scan pada arena yang sama, memahami arti resolusi, dan melihat efek `bodyexclude` secara nyata.

Simpan sebagai `p3_pola_scan.py` (memakai `arena.xml` dari Praktik 2) lalu jalankan.

```
import time
import numpy as np
import mujoco
from mujoco_lidar import MjLidarWrapper, scan_gen

model = mujoco.MjModel.from_xml_path('arena.xml')
data = mujoco.MjData(model)
robot_id = model.body('robot').id
mujoco.mj_forward(model, data)           # robot di (0, 0), site LiDAR di z =
0.35 m

# --- Bagian A: bandingkan pola scan ---
polanya = {
    'garis 360'      : scan_gen.create_lidar_single_line(360),
    'grid 64x16'     : scan_gen.generate_grid_scan_pattern(64, 16),
    'Velodyne HDL64' : scan_gen.generate_HDL64(),
    'Velodyne VLP32' : scan_gen.generate_vlp32(),
    'Ouster OS128'   : scan_gen.generate_os128(),
}
lidar = MjLidarWrapper(model, site_name='lidar_site', backend='cpu',
                       cutoff_dist=30.0, args={'bodyexclude': robot_id})

print(f'{"pola":16s} {"sinar":>8s} {"kena":>8s} {"waktu ms":>9s}  phi
min..max (derajat)')
for nama, (t, p) in polanya.items():
    t0 = time.perf_counter()
    r = lidar.trace_rays(data, t, p)
    ms = (time.perf_counter() - t0) * 1000
    print(f'{nama:16s} {len(t):8d} {int((r > 0).sum()):8d} {ms:9.2f}  '
          f'{np.degrees(p.min()):6.1f}..{np.degrees(p.max()):5.1f}')

# --- Bagian B: pengaruh bodyexclude ---
t, p = scan_gen.generate_grid_scan_pattern(64, 16)          # phi -60..+60
derajat
for excl in (None, robot_id):
    args = {} if excl is None else {'bodyexclude': excl}
    l = MjLidarWrapper(model, site_name='lidar_site', backend='cpu',
cutoff_dist=30.0, args=args)
    r = l.trace_rays(data, t, p)
    dekat = int(((r > 0) & (r < 0.3)).sum())
    print(f'bodyexclude={"ya " if excl is not None else "tidak"}: sinar
dengan jarak < 0.3 m = {dekat}')

# --- Bagian C: pengaruh resolusi ---
for kolom in (36, 180, 720):
```

```
    t, p = scan_gen.create_lidar_single_line(kolom)
    r = lidar.trace_rays(data, t, p)
    print(f'resolusi {kolom:4d} sinar/360 deg ->
{np.degrees(2*np.pi/kolom):5.2f} derajat/sinar, kena = {(r > 0).sum()}')
```

Keluaran dari uji saya (kolom "waktu" bergantung CPU Anda, jadi angkanya akan berbeda; kolom lain harus sama):

```
pola                sinar     kena  waktu ms  phi min..max (derajat)
garis 360             360      360      0.93     0.0..  0.0
grid 64x16           1024      582      1.19   -60.0.. 60.0
Velodyne HDL64     110016   110016     63.63   -24.9..  2.0
Velodyne VLP32     120000   105261     51.43   -25.0.. 20.0
Ouster OS128       260096   181947    129.68   -22.5.. 22.5
bodyexclude=tidak: sinar dengan jarak < 0.3 m = 248
bodyexclude=ya : sinar dengan jarak < 0.3 m = 0
resolusi   36 sinar/360 deg -> 10.00 derajat/sinar, kena = 36
resolusi  180 sinar/360 deg ->  2.00 derajat/sinar, kena = 180
resolusi  720 sinar/360 deg ->  0.50 derajat/sinar, kena = 720
```

Cara membaca hasil:

- **Jumlah sinar menentukan beban hitung.** Garis 360 hanya 360 sinar (kurang dari 1 ms), sedangkan OS128 punya sekitar 260 ribu sinar (sekitar 130 ms di sandbox saya). Waktu naik kira-kira sebanding dengan jumlah sinar.
- **"Kena" bukan selalu 100%.** Pada garis 360, semua sinar kena karena dinding mengelilingi robot. Pada grid 64×16 dengan elevasi sampai ±60°, hanya 582 dari 1024 yang kena: sinar yang mengarah ke atas keluar dari arena (tidak ada atap), sehingga bernilai -1.
- **Elevasi menentukan apa yang terlihat.** Rentang φ HDL64 sekitar -24,9° sampai +2° sehingga hampir semua sinarnya menunjuk ke lantai dan kena. Sinar ke atas pada pola lain meleset.
- **`bodyexclude` penting.** Tanpa itu, 248 sinar curam ke bawah mengenai badan robot sendiri pada jarak kurang dari 0,3 m. Dalam algoritma penghindar rintangan, titik palsu ini akan terbaca sebagai "rintangan sangat dekat".
- **Resolusi sudut menentukan detail.** Pada 10° per sinar, kotak 0,8 m di jarak 3 m (lebar sudut sekitar 15°) hanya tertangkap 1 sampai 2 sinar; pada 0,5° tertangkap sekitar 30 sinar. Angka "kena" di Bagian C sama dengan jumlah sinar karena dinding mengelilingi robot.

Versi mudah: makin banyak sinar, makin detail gambarnya, tetapi komputer makin lama menghitung (seperti foto beresolusi tinggi yang file-nya besar). Sinar yang mengarah ke langit tidak mengenai apa pun karena arena tidak beratap. Tanpa `bodyexclude`, robot

"melihat" badannya sendiri dan mengira ada benda sangat dekat. Makin kecil sudut antar sinar, makin kecil benda yang masih tertangkap.

Pola Livox (opsional). Livox memakai pola non-repetitif yang butuh modul tambahan (`mujoco_lidar.scan_gen_livox_ti`, berbasis Taichi). Contoh dari dokumentasi repo:

```
from mujoco_lidar.scan_gen_livox_ti import LivoxGenerator
livox = LivoxGenerator(pattern='mid360')
theta, phi = livox.sample_ray_angles()
```

Saya tidak menjalankan bagian ini karena membutuhkan Taichi. Jika Anda memasangnya (`pip install taichi`), hasilnya tetap berupa pasangan `(theta, phi)` yang bisa langsung dimasukkan ke `trace_rays`.

Eksperimen: buat pola grid dengan `theta_range=(-np.pi/4, np.pi/4)` dan `phi_range=(-np.radians(10), np.radians(10))` untuk meniru LiDAR depan dengan FoV 90° × 20°. Pakai 90 kolom dan 10 baris.

Praktik 4: Visualisasi dan analisis point cloud

Tujuan: mengubah jarak mentah menjadi empat tampilan yang dipakai praktisi: range image, tampak atas, histogram jarak, dan occupancy grid 2D. Dari sana Anda juga mencari titik terdekat, dasar dari penghindar rintangan.

Simpan sebagai `p4_visualisasi.py` lalu jalankan.

```
import numpy as np
import mujoco
import matplotlib
matplotlib.use('Agg')                    # hapus baris ini untuk jendela
interaktif
import matplotlib.pyplot as plt
from mujoco_lidar import MjLidarWrapper, scan_gen

model = mujoco.MjModel.from_xml_path('arena.xml')
data = mujoco.MjData(model)
robot_id = model.body('robot').id
mocap_id = model.body_mocapid[robot_id]

# Robot di (-2, -1) menghadap ke timur laut (45 derajat)
yaw = np.radians(45)
data.mocap_pos[mocap_id] = [-2.0, -1.0, 0.15]
data.mocap_quat[mocap_id] = [np.cos(yaw/2), 0, 0, np.sin(yaw/2)]
mujoco.mj_forward(model, data)
```

```
lidar = MjLidarWrapper(model, site_name='lidar_site', backend='cpu',
                       cutoff_dist=30.0, args={'bodyexclude': robot_id})

# LiDAR grid 180 kolom x 16 baris, elevasi -15..+15 derajat
KOLOM, BARIS = 180, 16
theta, phi = scan_gen.generate_grid_scan_pattern(
    KOLOM, BARIS, theta_range=(-np.pi, np.pi),
    phi_range=(-np.radians(15), np.radians(15)))
r = lidar.trace_rays(data, theta, phi)
valid = r > 0
print(f'sinar valid: {valid.sum()} dari {len(r)}')

# --- titik dunia ---
lokal = lidar.get_hit_points()[valid]
dunia = lokal @ np.asarray(lidar.sensor_rotation).T + lidar.sensor_position

# --- analisis 1: titik terdekat ---
d = r[valid]
k = int(np.argmin(d))
print(f'titik terdekat: {d[k]:.2f} m, di dunia = {np.round(dunia[k], 2)}')

# --- analisis 2: occupancy grid 2D (tampak atas), sel 10 cm ---
RES, UKURAN = 0.1, 8.4
n = int(UKURAN / RES)
grid = np.zeros((n, n), dtype=np.uint8)
ix = np.clip(((dunia[:, 0] + UKURAN/2) / RES).astype(int), 0, n-1)
iy = np.clip(((dunia[:, 1] + UKURAN/2) / RES).astype(int), 0, n-1)
mask_tinggi = dunia[:, 2] > 0.1           # abaikan titik lantai
grid[iy[mask_tinggi], ix[mask_tinggi]] = 1
print(f'sel terisi pada occupancy grid: {int(grid.sum())}')

# --- gambar ---
fig, ax = plt.subplots(1, 4, figsize=(20, 4.5))
img = np.where(valid, r, np.nan).reshape(BARIS, KOLOM)
im = ax[0].imshow(img, aspect='auto', cmap='viridis')
ax[0].set_title('Range image (baris=elevasi, kolom=azimut)');
fig.colorbar(im, ax=ax[0], label='jarak (m)')
ax[1].scatter(dunia[:, 0], dunia[:, 1], s=2, c=dunia[:, 2]);
ax[1].set_aspect('equal')
ax[1].plot(-2, -1, 'r^'); ax[1].set_title('Tampak atas (warna = tinggi)')
ax[2].hist(d, bins=40); ax[2].set_title('Histogram jarak');
ax[2].set_xlabel('m')
ax[3].imshow(grid, origin='lower', extent=[-UKURAN/2, UKURAN/2, -UKURAN/2,
```

```
UKURAN/2], cmap='gray_r')
ax[3].set_title('Occupancy grid (hitam = terisi)')
fig.tight_layout(); fig.savefig('p4_hasil.png', dpi=80)
print('gambar disimpan: p4_hasil.png')
```

Keluaran teks yang diharapkan (hasil uji saya):

```
sinar valid: 2372 dari 2880
titik terdekat: 1.35 m, di dunia = [-2.89 -1.96  0.  ]
sel terisi pada occupancy grid: 179
gambar disimpan: p4_hasil.png
```

Cara membaca empat gambar di `p4_hasil.png`:

Gambar	Cara membaca	Yang akan Anda lihat
Range image	Setiap piksel satu sinar; baris paling atas = elevasi terendah (sinar menunjuk ke bawah), baris paling bawah = elevasi tertinggi, kolom = azimut. Putih = sinar meleset (-1, diubah ke NaN)	Baris atas selalu berwarna karena sinar ke bawah mengenai lantai. Area putih di bagian bawah adalah sinar yang menunjuk ke atas dan lolos di atas dinding setinggi 1 m. Garis vertikal gelap = objek dekat (silinder, bola, kotak)
Tampak atas	Titik diproyeksikan ke bidang x-y; warna = tinggi z	Lingkaran konsentris di sekitar robot adalah titik di lantai dari sinar yang menunjuk ke bawah dengan sudut berbeda. Titik di tepi arena adalah dinding
Histogram jarak	Berapa banyak titik di tiap rentang jarak	Banyak titik lantai dekat (1,3 sampai 3 m), sedikit titik dinding jauh (4 sampai 8 m)
Occupancy grid	Sel 10 cm yang berisi titik di atas lantai ditandai terisi	Dinding dan rintangan terlihat sebagai garis dan gumpalan hitam; lantai tidak ikut karena disaring `z > 0.1`

Catatan penting: karena kita memakai LiDAR 3D, sebagian besar titik berasal dari lantai. Jika tidak disaring, lantai akan terbaca sebagai "rintangan" di peta 2D. Penyaringan `z > 0.1` adalah cara paling sederhana; pada praktik nyata, orang memakai estimasi bidang lantai.

Grid hanya menandai sel yang terkena sinar. Sel kosong di antara robot dan dinding berarti "tidak ada titik", bukan "pasti bebas"; peta yang benar-benar membedakan bebas dan tak diketahui butuh ray tracing di grid, di luar cakupan modul ini.

Versi mudah: *range image* adalah foto yang warnanya menunjukkan jarak. *Tampak atas* adalah denah dari atas. *Histogram* menghitung berapa banyak titik yang dekat dan yang jauh. *Occupancy grid* adalah peta kotak-kotak: kotak hitam berarti ada benda. Lingkaran-lingkaran di tampak atas adalah lantai yang terkena sinar miring ke bawah, jadi kita saring supaya lantai tidak dikira rintangan.

Eksperimen:

- Ganti `phi_range` menjadi `(-np.radians(2), np.radians(2))` dan lihat gambar tampak atas. Apa yang hilang?
- Ubah pose robot ke (3, 3) dengan yaw 180° dan perkirakan titik terdekat sebelum menjalankan skrip.
- Gunakan `scan_gen.generate_HDL64()` sebagai pengganti grid, lalu buat range image. Perintah `reshape(16, 180)` tidak akan cocok. Bagaimana cara membentuknya? (Jawaban di Latihan.)

Praktik 5: LiDAR pada robot bergerak di arena

Tujuan: memakai LiDAR sebagai "mata" robot. Robot memindai di setiap langkah, lalu memutuskan maju atau belok agar tidak menabrak rintangan. Ini siklus dasar sense, decide, act.

5.1 Rancangan

1. **Sense:** scan setengah lingkaran di depan robot (-90° sampai +90°, 181 sinar).
2. **Decide:** jika ada benda kurang dari 0,8 m dalam kerucut ±30° di depan, putar ke sisi yang rata-rata jaraknya lebih besar (kiri atau kanan). Jika tidak, maju.
3. **Act:** perbarui pose robot (x, y, yaw) lalu taruh di scene lewat mocap.

Robot ini kinematik (posisinya kita atur langsung), jadi kita fokus pada LiDAR, bukan pada fisika roda atau kontrol motor.

5.2 Kode

Simpan sebagai `p5_robot_reaktif.py` lalu jalankan.

```
import numpy as np
import mujoco
import matplotlib
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
from mujoco_lidar import MjLidarWrapper, scan_gen

model = mujoco.MjModel.from_xml_path('arena.xml')
data = mujoco.MjData(model)
robot_id = model.body('robot').id
mocap_id = model.body_mocapid[robot_id]
mujoco.mj_forward(model, data)

JARAK_MAKS = 10.0
lidar = MjLidarWrapper(model, site_name='lidar_site', backend='cpu',
                       cutoff_dist=JARAK_MAKS, args={'bodyexclude':
robot_id})

# LiDAR 2D: setengah lingkaran di depan robot (-90..+90 derajat), 181 sinar
theta, phi = scan_gen.create_lidar_single_line(181, horizontal_fov=np.pi)

# Parameter gerak
x, y, yaw = -3.0, -3.0, 0.0       # pose awal
V, W, DT = 0.8, 1.2, 0.05         # m/s, rad/s, detik per langkah
AMBANG_DEPAN = 0.8                # meter: kurang dari ini = belok
DURASI = 60.0                     # detik simulasi

jejak, jarak_min = [], []
tabrakan = 0
for _ in range(int(DURASI / DT)):
    # 1) taruh robot di pose saat ini
    data.mocap_pos[mocap_id] = [x, y, 0.15]
    data.mocap_quat[mocap_id] = [np.cos(yaw/2), 0, 0, np.sin(yaw/2)]
    mujoco.mj_forward(model, data)

    # 2) scan dan ganti -1 (meleset) dengan jarak maksimum
    r = lidar.trace_rays(data, theta, phi)
    rr = np.where(r > 0, r, JARAK_MAKS)

    # 3) ringkas: jarak ke depan, dan rata-rata jarak kiri vs kanan
    depan = rr[np.abs(theta) < np.radians(30)].min()
    kiri  = rr[theta >  np.radians(30)].mean()
    kanan = rr[theta < -np.radians(30)].mean()

    # 4) keputusan: ada rintangan di depan -> putar ke sisi yang lebih
lapang; kalau tidak -> maju
    if depan < AMBANG_DEPAN:
        yaw += (W if kiri > kanan else -W) * DT
    else:
        x += V * np.cos(yaw) * DT
```

```
        y += V * np.sin(yaw) * DT

    jejak.append((x, y))
    jarak_min.append(rr.min())
    if rr.min() < 0.25:
        tabrakan += 1

print(f'jarak terdekat ke rintangan selama run: {min(jarak_min):.2f} m')
print(f'langkah dengan jarak < 0.25 m (hampir tabrak): {tabrakan}')
print(f'posisi akhir: ({x:.2f}, {y:.2f}); masih di dalam arena: {abs(x) < 4
and abs(y) < 4}')

# gambar lintasan di atas denah
jejak = np.array(jejak)
fig, ax = plt.subplots(figsize=(6, 6))
ax.plot(jejak[:, 0], jejak[:, 1], '-', lw=1)
ax.plot(*jejak[0], 'go', label='awal'); ax.plot(*jejak[-1], 'rx',
label='akhir')
for nama, (cx, cy, rad) in {'kotak': (2, 1, 0.4), 'silinder': (-1.5, 2, 0.3),
'bola': (1, -2.5, 0.35)}.items():
    ax.add_patch(plt.Circle((cx, cy), rad, color='orange', alpha=0.6))
ax.set_xlim(-4, 4); ax.set_ylim(-4, 4); ax.set_aspect('equal');
ax.grid(True); ax.legend()
ax.set_title('Lintasan robot reaktif (kotak digambar sebagai lingkaran)')
fig.savefig('p5_lintasan.png', dpi=100)
```

Keluaran yang diharapkan (hasil uji saya):

```
jarak terdekat ke rintangan selama run: 0.40 m
langkah dengan jarak < 0.25 m (hampir tabrak): 0
posisi akhir: (-3.23, 3.59); masih di dalam arena: True
```

5.3 Penjelasan

- **Mengapa `theta` dari -90° sampai +90°?** `create_lidar_single_line(181, horizontal_fov=np.pi)` membuat sudut dari -π/2 sampai +π/2. Sudut 0 = arah depan robot, positif = kiri. Karena site ikut berputar bersama body, "depan" selalu mengikuti yaw robot.
- **Mengapa -1 diganti `JARAK_MAKS`?** Sinar yang meleset artinya "tidak ada apa-apa dalam jangkauan", yaitu ruang bebas. Jika dibiarkan -1, `min()` akan mengira ada rintangan di jarak -1.
- **Mengapa rata-rata kiri vs kanan?** Itu aturan sederhana untuk memilih sisi yang lebih lapang. Aturan ini bisa terjebak di sudut tertentu; menyempurnakannya adalah latihan.

- **Batasan LiDAR 2D:** sensor setinggi 0,35 m hanya melihat rintangan yang menjangkau ketinggian itu. Ketiga rintangan di arena ini memenuhi syarat; rintangan rendah (misalnya tinggi 0,2 m) tidak akan terlihat.

Versi mudah: robot berjalan seperti orang di ruangan gelap sambil mengayunkan tongkat: ada tembok dekat di depan? Kalau ya, belok ke sisi yang lebih lapang. Kalau tidak, lanjut maju. Ulangi terus setiap 0,05 detik. Nilai -1 diganti jarak maksimum karena artinya "kosong sejauh jangkauan", bukan "ada benda di jarak -1".

5.4 Opsional: melihat robot bergerak secara langsung

Jika komputer Anda punya layar, bungkus loop dengan viewer pasif MuJoCo. Kerangka ini tidak saya jalankan di sandbox (butuh display), jadi anggap sebagai titik awal:

```
import time
import mujoco.viewer

with mujoco.viewer.launch_passive(model, data) as viewer:
    while viewer.is_running():
        # ... isi satu iterasi loop dari 5.2 (langkah 1 sampai 4) ...
        viewer.sync()
        time.sleep(DT)
```

5.5 Mengembangkan lebih lanjut

- Ganti LiDAR garis dengan grid 3D, lalu saring titik lantai (`z > 0.1`) seperti di Praktik 4.
- Tambahkan noise Gaussian pada `r` (mis. `r + np.random.normal(0, 0.02, r.shape)`) untuk mensimulasikan sensor nyata, lalu cek apakah robot tetap aman.
- Pelajari contoh robot asli di repo: `examples/unitree_go2.py` dan `examples/unitree_g1.py` (mendukung `--backend cpu`).

Latihan mandiri dan kunci jawaban

Kerjakan dulu tanpa melihat kunci. Jawaban numerik di bawah sudah saya cocokkan dengan hasil eksekusi kode di sandbox (backend CPU, mujoco-lidar 0.3.5).

A. Pemahaman konsep

No	Soal	Kunci jawaban
A1	Pantulan laser kembali setelah 40 ns. Berapa jarak objek?	d = (3×10^8 × 40×10^-9) / 2 = 6 m

No	Soal	Kunci jawaban
A2	Mengapa LiDAR 64 channel memberi gambaran vertikal lebih rapat daripada 16 channel?	Pada FoV vertikal yang sama, 64 laser berarti selisih sudut φ antar-laser lebih kecil, jadi lebih banyak baris titik per tinggi objek
A3	Mengapa `trace_rays` bisa mengembalikan -1 dan apa artinya?	Sinar tidak mengenai geom apa pun dalam `cutoff_dist`. Nilai itu harus disaring (`r > 0`) sebelum dihitung atau diplot
A4	Mengapa kode di Praktik 1 menulis `backend='cpu'` eksplisit?	Nilai bawaan konstruktor adalah `'taichi'`, yang butuh paket Taichi dan idealnya GPU
A5	Apa beda `get_hit_points()` dengan koordinat dunia?	`get_hit_points()` ada di bingkai sensor. Dunia = `lokal @ sensor_rotation.T + sensor_position`
A6	Sebutkan dua hal yang tidak dimodelkan simulasi ini dibanding sensor nyata	Noise pengukuran dan intensitas pantulan (juga efek kaca dan cuaca). Lihat tabel di Bagian 2.2

B. Eksperimen di tiap praktik

No	Soal	Kunci jawaban (hasil eksekusi)
B1	Praktik 1: kotak dipindah ke `x=4`. Berapa jarak ke depan dan berapa sinar kena?	3,50 m (sisi depan di x = 3,5) dan 16 sinar kena. Lebar sudut kotak sekitar 2 × arctan(0,5/3,5) ≈ 16°
B2	Praktik 1: `cutoff_dist=1.0`	0 sinar kena, karena kotak berada di 1,5 m, di luar jangkauan
B3	Praktik 2: kotak baru `size='0.3 0.3 0.3'` di (-2, -1), robot di (-3, -1) menghadap +x	Jarak ke depan 0,70 m (sisi depan kotak di x = -2,3; sensor di x = -3)
B4	Praktik 3: FoV depan 90° × 20°	Lihat kode di bawah tabel; hasilnya 900 sinar
B5	Praktik 4: `phi_range` ±2°	Lingkaran titik lantai hilang. Pada uji saya, seluruh 2880 sinar mengenai dinding atau rintangan dengan tinggi titik antara 0,08 dan 0,62 m

No	Soal	Kunci jawaban (hasil eksekusi)
B6	Praktik 4: robot di (3, 3) menghadap 180°	Titik terdekat 1,00 m (dinding utara dan dinding timur sama-sama berjarak 1 m dari (3, 3))
B7	Praktik 4: range image untuk HDL64	Pada HDL64, tiap 64 sinar berurutan punya θ sama dengan 64 nilai φ berbeda (110016 = 64 × 1719). Jadi bentuknya `r.reshape(-1, 64).T`, yaitu (64 baris, 1719 kolom). Urutan ini berbeda dengan grid yang berurutan per baris, makanya `reshape(16, 180)` salah untuk pola ini

Kode B4:

```
theta, phi = scan_gen.generate_grid_scan_pattern(
    90, 10,
    theta_range=(-np.pi/4, np.pi/4),
    phi_range=(-np.radians(10), np.radians(10)))
print(len(theta))   # 900
```

C. Tantangan

C1. Ketahanan terhadap noise. Ubah Praktik 5 agar jarak LiDAR diberi noise Gaussian: `rr = rr + np.random.normal(0, SIGMA, rr.shape)`. Uji `SIGMA` = 0, 0,02, dan 0,2 m. Ukur jarak minimum asli (tanpa noise) ke rintangan dan posisi akhir.

Kriteria lulus: jarak minimum asli tetap di atas 0,25 m dan robot tetap di dalam arena. Hasil uji saya (seed 0): jarak minimum asli 0,40 m (σ=0), 0,41 m (σ=0,02), dan 0,55 m (σ=0,2); tidak ada langkah di bawah 0,25 m, dan robot selalu tetap di dalam arena. Artinya aturan sederhana ini cukup tahan terhadap noise sebesar itu pada arena ini. Angkanya bisa berbeda jika Anda mengubah seed, aturan, atau tata letak.

C2. Rintangan rendah. Tambahkan balok setinggi 0,2 m di jalur robot. Apakah LiDAR garis di z = 0,35 m melihatnya? Jawab dengan menjalankan, lalu perbaiki dengan grid 3D yang memiliki sinar miring ke bawah.

C3. Peta sendiri. Gabungkan occupancy grid dari Praktik 4 dengan lintasan Praktik 5: akumulasikan titik dunia dari scan di setiap langkah (bukan satu kali) sehingga peta terisi saat robot berkeliling.

C4. Bandingkan pola. Jalankan Praktik 5 dengan scan grid kecil 90×10 (B4). Bandingkan jumlah langkah yang "hampir tabrak" dan waktu komputasi dengan garis 181 sinar.

D. Rubrik penilaian mandiri

Kemampuan	Tanda Anda sudah menguasainya
Konsep	Bisa menjelaskan ToF, ray casting, dan arti θ, φ, r tanpa melihat catatan
Praktik dasar	Praktik 1 sampai 3 berjalan dan keluarannya cocok dengan yang diharapkan
Analisis	Bisa menjelaskan mengapa lantai muncul di tampak atas dan cara menyaringnya
Integrasi	Praktik 5 berjalan, dan Anda bisa mengubah ambang atau aturan lalu memprediksi akibatnya

Troubleshooting

Baris bertanda "diuji" menunjukkan pesan error yang saya reproduksi langsung; sisanya adalah penyebab umum berdasarkan perilaku library di atas.

Gejala	Penyebab	Solusi
`ImportError: Failed to import Taichi backend dependencies...` (diuji)	Anda tidak menulis `backend='cpu'`, sehingga library memakai backend bawaan `taichi` yang belum terpasang	Tulis `backend='cpu'` di `MjLidarWrapper(...)`. Atau pasang Taichi (`pip install "mujoco-lidar[taichi]"`) jika memang ingin backend GPU
`KeyError: Invalid name 'salah'. Valid names: [...]` (diuji; muncul saat `trace_rays`, bukan saat membuat objek)	Nama `site_name` tidak ada di model	Cocokkan dengan atribut `name` pada `<site>`; pesan error menampilkan daftar nama yang valid
`ModuleNotFoundError: No module named 'mujoco_lidar'`	Lingkungan virtual tidak aktif, atau `pip install` dijalankan di Python lain	Aktifkan `.venv`, lalu `python -m pip install mujoco-lidar` agar pip dan Python sama

Gejala	Penyebab	Solusi
Skrip aktivasi venv diblokir di PowerShell	Kebijakan eksekusi Windows	Jalankan `Set-ExecutionPolicy -Scope Process RemoteSigned` di jendela itu, lalu aktifkan ulang
Semua jarak bernilai -1	Tidak ada geom dalam jangkauan: `cutoff_dist` terlalu kecil, sinar sejajar lantai, site di luar arena, atau arah sinar tidak menuju objek	Periksa posisi site (`lidar.sensor_position`), naikkan `cutoff_dist` , atau tambahkan elevasi pada pola scan
Jarak tidak berubah walau robot dipindah	`mj_forward` tidak dipanggil setelah mengubah `mocap_pos` atau `qpos` , sehingga posisi site belum diperbarui	Panggil `mujoco.mj_forward(model, data)` sebelum `trace_rays`
Banyak titik palsu sangat dekat dengan sensor	Sinar mengenai badan robot sendiri	Pakai `args={'bodyexclude': model.body('robot').id}` (lihat Praktik 3, bagian B)
Titik muncul di (0, 0, 0) pada plot	Baris `get_hit_points()` untuk sinar yang meleset bernilai nol	Saring dengan `hit = r > 0` lalu pakai `pts[hit]`
Point cloud terlihat miring atau tidak sesuai arena	Titik masih di bingkai sensor, belum diubah ke dunia	`dunia = lokal @ np.asarray(lidar.sensor_rotation).T + lidar.sensor_position`
Lantai terbaca sebagai rintangan di peta 2D	LiDAR 3D mengenai lantai	Saring titik rendah (`z > 0.1`)
Jendela matplotlib atau viewer tidak muncul / error display	Server atau WSL tanpa layar, atau backend matplotlib tidak cocok	Pakai `matplotlib.use('Agg')` dan simpan ke PNG (seperti di modul ini). Viewer MuJoCo butuh display

Gejala	Penyebab	Solusi
Scan terasa lambat	Terlalu banyak sinar untuk CPU (mis. OS128 sekitar 260 ribu sinar)	Kurangi sinar (grid atau garis), atau pakai backend GPU (`taichi` , `warp` , `jax`)
Perbedaan hasil dengan contoh di repo	Repo terus berubah, dan beberapa contoh lama memakai pola pemanggilan berbeda (mis. `get_hit_points()` setelah `trace_rays`)	Cek `help(MjLidarWrapper)` pada versi yang terpasang dan rujuk API.md

Jika masalah tidak terselesaikan, buka issue di repo atau periksa INSTALLATION.md. Sertakan versi Python, versi `mujoco` dan `mujoco_lidar` (keluaran `p0_cek.py`), serta pesan error lengkap.

Referensi dan ringkasan file

Daftar file di folder `lidar_lab`

File	Dibuat di	Fungsi
`p0_cek.py`	Bagian 3	Cek versi dan fungsi yang tersedia
`p1_scan_pertama.py`	Praktik 1	Scan satu garis pada scene string; verifikasi rumus titik 3D
`arena.xml`	Praktik 2	Arena 8 m × 8 m, tiga rintangan, robot mocap ber-site LiDAR
`p2_arena.py`	Praktik 2	Scan dari dua pose; gambar tampak atas
`p3_pola_scan.py`	Praktik 3	Perbandingan pola scan, `bodyexclude` , dan resolusi
`p4_visualisasi.py`	Praktik 4	Range image, tampak atas, histogram, occupancy grid
`p5_robot_reaktif.py`	Praktik 5	Robot yang menghindari rintangan dengan LiDAR

Lingkungan uji

Seluruh skrip di atas dijalankan oleh saya di Linux (sandbox) dengan Python 3.12.3, MuJoCo 3.15.0, `mujoco-lidar` 0.3.5, backend CPU, tanpa layar. Bagian yang butuh layar (viewer), Taichi, atau clone repo tidak diuji dan sudah ditandai di tempatnya. Versi library di komputer Anda mungkin lebih baru; jika ada beda perilaku, rujuk dokumentasi resmi.

Sumber

Halaman berikut saya buka langsung saat menyusun modul:

- README repo MuJoCo-LiDAR: fitur, backend, kecepatan, instalasi, daftar contoh.
- USAGE.md: contoh pemakaian wrapper, pola scan, `bodyexclude`, `geomgroup`.
- API.md: tanda tangan `MjLidarWrapper`, `trace_rays`, generator pola scan, opsi backend.
- examples/example_mjcf.py: pola viewer plus plot point cloud real-time dengan thread.

Di luar itu, INSTALLATION.md tertaut dari README tetapi tidak saya buka.

Data tanggal: 12 Oktober 2026. Angka kecepatan backend berasal dari README dan bergantung perangkat keras; angka waktu di Praktik 3 berasal dari sandbox saya.

Langkah belajar lanjutan

1. Jalankan `examples/example_mjcf.py` dan `examples/unitree_go2.py --backend cpu` dari repo dan bandingkan dengan Praktik 5.
2. Coba backend GPU (`warp` atau `taichi`) pada pola OS128 dan ukur peningkatan kecepatannya.
3. Eksplorasi pola Livox (`LivoxGenerator`) dan bandingkan point cloud-nya dengan HDL64.
4. Jika tertarik ROS, repo menyediakan contoh ROS1 dan ROS2 (`examples/lidar_vis_ros2.py`, dst.).